Type conversions

26 April 2026 17:44

```
> "1" + "2"
'12'
> "1" + 2
'12'
> 1 + "2"
'12'
> "1" * "2"
2
> "1" / "2"
0.5
> "5" % 3
2
> "5" - "3"
2
> "5" + parseInt("3")
'53'
> parseInt("5") + parseInt("3")
8
```

concatenation.

string

int [concatenation overrides addition]

here JS converts the values int numeric form implicitly.

implicit conversion.

explicit conversion: In explicit conversion we forcefully convert data into particular type or our desired type, as per our requirement.

```
> let value = true;
undefined
> typeof(value)
'boolean'
> value = String(value)
'true'
> typeof(value)
'string'
>
```

```
> Number
[Function: Number]
```

```
> '23.5' + '7.8'
'23.57.8'
> Number('23.5') + Number('7.8')
31.3
```

Explicit conversion is usually required when we read a value from a string based sources, like a text form but expect a number to be entered.

Value	Becomes
undefined	> Number(undefined) NaN

null	> Number(null) 0
true and false	> Number(true) 1 > Number(false) 0
string	White spaces (Include spaces, tabs \t, new lines \n etc.) From the start and end are removed. If the remaining string is empty, the result is zero. Otherwise. The number is "Read" from the string. And error gives NaN. > Number('ten') NaN > Number('') 0 > Number(' ') 0 > Number('\t') 0 > Number('\n') 0

Boolean Conversion

Boolean conversion is the simplest one.
It happens in logical operations.
The conversion rule is:

- Values are intuitively "empty" like 0 an empty string, null, undefined and Nan becomes false.
- Other values become true.

```
> Boolean
[Function: Boolean]

> Boolean(1)
true
> Boolean(0)
false
> Boolean('hello')
true
> Boolean('')
false
> Boolean(undefined)
false
> Boolean(null)
false
> Boolean(NaN)
false
>
```